1 Vulnerability of the Initial Scheme

Current analysis indicates that the proposed scheme is highly insecure. The fundamental flaw lies in the fact that knowing merely a single ciphertext-plaintext pair allows an adversary to reconstruct the unity roots and subsequently decrypt the entire system. This attack can be executed as follows.

Given a known pair (c, m) where $m \neq 0$: One can exponentiate c up to n times and construct a system of linear equations (SLE). The exponentiations of c are linearly independent. Furthermore, possessing multiple plaintext-ciphertext pairs also satisfies this condition. The resulting SLE takes the form $A \cdot x = b$:

$$\begin{bmatrix} — & c^1 & — \\ — & c^2 & — \\ — & \dots & — \\ — & c^n & — \end{bmatrix} \cdot x = \begin{bmatrix} m^1 \\ m^2 \\ \dots \\ m^n \end{bmatrix} \tag{1}$$

This system can be solved for x. Assuming m is located at the i-th position in the encoded vector, x will consequently represent the i-th column of the Discrete Fourier Transform (DFT) matrix. This directly exposes the unity roots, allowing for the complete reconstruction of the DFT matrix.

1.1 Modular Inversion vs. Square Roots

A critical question arises: Can inverse elements modulo N be found without knowing the prime factors p and q?

- Yes, this is achievable using the Extended Euclidean Algorithm. Therefore, Gaussian elimination is viable, and SLEs can be solved. Complex numbers can also be normalized, and their inverses can be found.
- A common misconception is that finding the inverse of a complex number modulo N requires calculating a square root. For a number $z = (a, b)$, the inverse is often expressed as $z^{-1} = \sqrt{|z|}^{-1} \cdot (a, -b)$, because $z^{-1} \cdot z = \sqrt{|z|}^{-1} \cdot (a, -b) \cdot (a, b) = \sqrt{|z|}^{-1} \cdot (a^2 + b^2, 0)$.
- However, calculating a square root is *not* necessary. To perform division, one only needs to compute the modular inverse of the norm $(a^2 + b^2)$, which can be done efficiently without knowing p and q. For example, modulo 11: The inverse of $(2, 3)$ is $(2, -3) \cdot (2^2 + 3^2)^{-1} \equiv (2, -3) \cdot 13^{-1} \equiv (2, -3) \cdot 2^{-1} \equiv (2, -3) \cdot 6 \equiv (1, -7) \equiv (1, 4)$. Proof: $(2, 3) \cdot (1, 4) = (2 \cdot 1 - 3 \cdot 4, 3 \cdot 1 + 2 \cdot 4) = (1, 0)$.

Conclusion: Division is strictly possible, rendering Gaussian elimination a viable attack vector.

2 Exploration of Alternative Encodings

2.1 Alternating Zero-Encoding

One potential mitigation could be encoding the message in an alternating pattern, e.g., $[m, 0, m, \dots]$. This would ensure that different powers of c are linearly dependent, thereby preventing Gaussian elimination. Consequently, c would simply be a specific vector scaled by m.

However, this approach is flawed:

- While it prevents Gaussian elimination, it remains questionable whether it sufficiently protects the unity roots.
- Checksums cannot be integrated, leading to a 1:1 correspondence between ciphertext and plaintext.

- It is structurally insecure: Knowing one pair reveals the scalar factor, instantly compromising all other elements.

2.2 Phase Correction and Length Encoding

Is it feasible to encode numbers such that their exact values remain completely unknown, thus inherently preventing Gaussian elimination? An idea is to encode the value not as $(m, 0)$, but as a complex coordinate (a, b), where $m = |(a, b)|$. A challenge here is that for additions, both numbers must share the same phase. To address this:

- Select a unity root u_Z of degree n.
- Encode the messages as $z = m \cdot u_Z$.
- Addition: $z + z' = u_Z(m) + u_Z(m')$.
- Multiplication: $z \cdot z' = (u_Z \cdot z_{Z'}) \cdot u_K$.

This approach fails because u_K, used to correct the phase, could be applied directly to u_Z. This would result in a number with an imaginary part of 0, directly exposing the length encoding.

3 Fundamental Doubts on Modular Arithmetic for FHE

After these counterexamples, a fundamental question emerges: Is it mathematically possible to construct a secure FHE scheme based purely on modular arithmetic? The inherent linearity implies that Gaussian elimination will almost always be applicable. This necessitates a formal proof regarding the limitations of these algebraic structures.

4 New Proposed Approaches

4.1 Vector-Sum Encoding

Instead of encoding elements within a vector, values could be encoded *as* a vector: $m \to m_1 \mid m_2 \mid m_3$, such that $m = m_1 + m_2 + m_3$. Evaluating this requires vector addition for homomorphic addition, and convolution (or element-wise multiplication post-FFT) for homomorphic multiplication. Since the individual components are random, an adversary cannot form an SLE.

Vulnerability: Multiplying this vector with the DFT matrix reveals $c[0] = \text{scale} \cdot (m_1 + m_2 + m_3)$. Because the first row of the DFT matrix consists entirely of $(1, 0)$, the plaintext value m is directly exposed at $c[0]$. Thus, no actual encryption takes place.

4.2 Complex Pairs (z, z') Encoding

Certain prime numbers allow for special complex pairs (z, z') that satisfy $z + z' = (1, 0)$ and $z \cdot z' = (0, 0)$. The message could be encoded as:

$$m \cdot z \mid \cdots \mid m \cdot z' \implies m = m \cdot z + m \cdot z' = m(z + z') = m \cdot (1, 0) \tag{2}$$

Calculating z and z' presumably requires knowledge of the prime factorization (PFZ). Since this is unknown to the adversary, the values shield the message, thus preventing the formulation of an SLE.

The Null-Space Vulnerability: An issue arises if $m = 0$, as the entries become known. The initial assumption was that knowing a zero-vector would not compromise the unity roots, as any scalar multiple $k \cdot x'$ of a non-trivial solution x' is also a solution, yielding no actionable information.

However, this assumption is incorrect. A zero in the plaintext *does* leak the unity roots. The null-space can be calculated via Gaussian elimination. The solution space is:

$$k \cdot x' = [k \cdot u^1, k \cdot u^2, \ldots, k \cdot u^n] \tag{3}$$

From this, one can trivially compute $u = u^1 = (k \cdot u^2)/(k \cdot u^1)$. The unknown scalar k cancels out completely. Using a composite n (e.g., $n = 2 \cdot 3 = 6$) and placing m at indices 0 and 3 does not offer sufficient protection, as exposing even a subset of roots is enough to partially decode the ciphertext.

4.3 Further Encoding Concepts

- **Shifted Length Encoding:** Use z, z' to perform length-encoding while preventing plaintext zeros. Checksums would be handled via modular arithmetic: $c := z \cdot [m_1, m_2, m_3] + z' \cdot [m_2, m_3, m_1]$ (e.g., using a shift). This prevents the length from being directly leaked, though new vulnerabilities may arise.

- **Pure (z, z') Encryption:** Abandon the FFT entirely and rely solely on the difficulty of finding the z, z' pairs without knowing the prime factorization.

- **Generator-Multiplied Length Encoding:** Multiply the length encoding by a generator to keep convolutions invariant: $c := [m_0 g^0, m_1 g^1, \ldots, m_n g^n]$ such that $m = \sum m_i$. An FFT can still be applied optionally to optimize multiplication complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$, which is equivalent to performing the FFT and applying z, z' afterward.